

SMART AGRICULTURE SYSTEM WITH EDGE COMPUTING AND DATA ANALYSIS

Supervisor: Diep Thanh Dang

Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology
Vietnam National University Ho Chi Minh City

April 07th 2026

Table of Contents

- 1 Introduction
- 2 System Architecture Overview
- 3 Requirements analysis
- 4 Software design
- 5 Implementation and Integration
- 6 Conclusion

Table of Contents

- 1 Introduction
- 2 System Architecture Overview
- 3 Requirements analysis
- 4 Software design
- 5 Implementation and Integration
- 6 Conclusion

Objectives & Scope

Project Objectives

- Implement a smart agriculture model to optimize labor, conserve resources, and address climate change challenges.
- Collect real-time environmental metrics, specifically temperature, air humidity, and light intensity.
- Automate essential agricultural mechanisms, such as irrigation and ventilation, based on predefined environmental thresholds.
- Provide an intuitive web-based dashboard allowing users to monitor and control the system remotely.

Project Scope

- Focus on a small-scale smart greenhouse infrastructure powered by the Yolo:Bit edge controller and various sensors (DHT20, LDR, IR).
- Establish a reliable cloud-connected system utilizing the MQTT protocol via Adafruit IO for seamless edge-to-cloud data transmission.

Key Features & Stakeholders

Core System Features

- **Dual Operation Modes:** Seamlessly switch between Auto Mode for condition-based automated responses and Manual Mode for direct user overrides.
- **Data Analysis & Alerts:** Subscribes to telemetry data to track trends (e.g., drying patterns) for proposing optimal watering schedules, while continuously monitoring for anomalies to trigger real-time notifications.
- **Scalable Architecture:** Built with software engineering best practices, incorporating Load Balancing for HTTP requests, Docker containerization, and structural Design Patterns (Observer, Strategy, Singleton).

Target Stakeholders

- **Farm Owners:** The primary users who require a reliable system and an easy-to-use interface to reduce the risk of crop damage.
- **System Technicians:** Administrators who rely on detailed system logs, easy configuration, and efficient troubleshooting workflows.

Table of Contents

- 1 Introduction
- 2 System Architecture Overview**
- 3 Requirements analysis
- 4 Software design
- 5 Implementation and Integration
- 6 Conclusion

The IoT Model

Hierarchical IoT Architecture

- Designed for seamless two-way communication between the physical layer and the user interface.

Bidirectional Data Flow

- **Uplink (Telemetry):** Edge Devices (Yolo:Bit) → Adafruit IO (MQTT Broker) → Load Balancer Subscriber → Backend Servers → Frontend Dashboard.
- **Downlink (Control):** Backend API → Adafruit IO (MQTT Publish) → Edge Gateway → Physical Actuators (Fans, Pumps, LEDs).

Core Middleware: Adafruit IO

- Acts as the centralized MQTT Broker organizing data into scalable "Feeds".
- Utilizes a lightweight Publish/Subscribe model, minimizing bandwidth and power consumption at the edge layer.

Table of Contents

- 1 Introduction
- 2 System Architecture Overview
- 3 Requirements analysis**
- 4 Software design
- 5 Implementation and Integration
- 6 Conclusion

Functional Requirements

System Control & Configuration

- **Access Management:** Secure user authentication for dashboard access.
- **Threshold Settings:** Ability to configure both **Ideal** ranges (for automatic actuator control) and **Critical ranges** (for emergency system overrides).
- **Actuator Control:** Support for **Auto Mode** (condition-based execution) and **Manual Mode** (direct user overrides via the dashboard).

Monitoring & Data History

- **Real-time Tracking:** Live monitoring of temperature, humidity, and light intensity on the dashboard and local LCD.
- **Historical Insights:** Historical environmental charts, actuator activity logs, and alert history.

Automated Alert System

- **Environmental & Security Alerts:** Notifications for metric deviations, critical emergencies, and IR sensor intrusion detection.
- **System Health Alerts:** Warnings for unresponsive actuators, abnormal sensor values (e.g., humidity = 0%), or Edge device connection loss.

Non-Functional Requirements

Performance Metrics

- **Execution Latency:** Manual control commands must be executed by actuators in $< 1s$.
- **Alert Delivery:** Notifications delivered within 5s of incident detection.
- **Sampling Rate:** Edge device data sampling interval must be $\leq 10s$.

Scalability & Reliability

- **High Availability:** Core server services must maintain an uptime of $\geq 99\%$.
- **Concurrency:** System must flawlessly handle data streams from at least 50 concurrent sensor nodes.

Security & Maintainability

- **Data Protection:** Encrypted data transmission between Edge devices and the server.
- **System Deployment:** Backend services containerized using Docker for consistent and reliable deployment.
- **Data Retention:** Environmental and activity logs stored for at least 1 year to support trend analysis.
- **Usability:** Intuitive UI ensuring core operations take no more than 3 steps.

Table of Contents

- 1 Introduction
- 2 System Architecture Overview
- 3 Requirements analysis
- 4 Software design**
- 5 Implementation and Integration
- 6 Conclusion

MVC-Inspired Architecture

- Models core control logic: controllers, services, entities, and views.
- Applies key design patterns: **Observer** and **Strategy**.

Design Focus

- Emphasizes real-time environmental monitoring and actuator control (Auto & Manual modes).
- Intentionally excludes secondary features (historical data, analysis) to maintain clarity on the primary control flow.

Software Design - Class Diagram

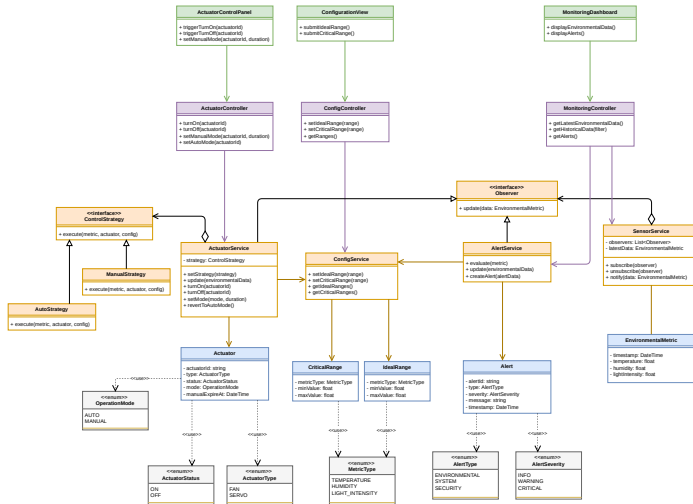


Figure 1: Class Diagram

Applied Design Patterns

- **Observer Pattern:** Enables real-time data propagation with loose coupling. `SensorService` acts as the subject, while `AlertService` and `ActuatorService` are observers reacting independently to new data.
- **Strategy Pattern:** Encapsulates control behaviors. The `ActuatorService` switches dynamically between `AutoStrategy` (evaluating against ideal/critical ranges) and `ManualStrategy` (user-defined override).
- **Singleton Pattern:** Ensures a single, shared instance for infrastructure components like database connections and the MQTT client.

Polyglot Persistence Strategy

- Designed to handle high-frequency IoT data and complex state management.

1. Time-Series DB (e.g., InfluxDB) for Telemetry

- Handles continuous streams of sensor data (temperature, humidity, light_intensity).
- Facilitates fast write operations and built-in downsampling for dashboards.

2. Relational DB (e.g., PostgreSQL/MySQL) for State and Logic

- Maintains system state, operational configurations, and user management.
- Core entities: Actuator, Configuration, ActuatorLog, User, and Alert.
- Ensures ACID compliance and referential integrity.

MQTT Communication (Edge to Cloud)

- Uses Adafruit IO MQTT Broker. All topics adhere to the naming convention: `<username>/feeds/<feed_name>`.

Telemetry Publishing (Edge → Cloud)

- `.../feeds/temp-test1`: Temperature data (e.g., "32.5")
- `.../feeds/humid-test1`: Air humidity data (e.g., "75.0")
- `.../feeds/luminor-test1`: Light intensity data (e.g., "850")

Command Subscription (Cloud → Edge)

- `.../feeds/actuator-fan`: Fan state ("ON" or "OFF")
- `.../feeds/actuator-pump`: Pump state ("ON" or "OFF")
- `.../feeds/actuator-led`: RGB LED state ("RED" or "GREEN")

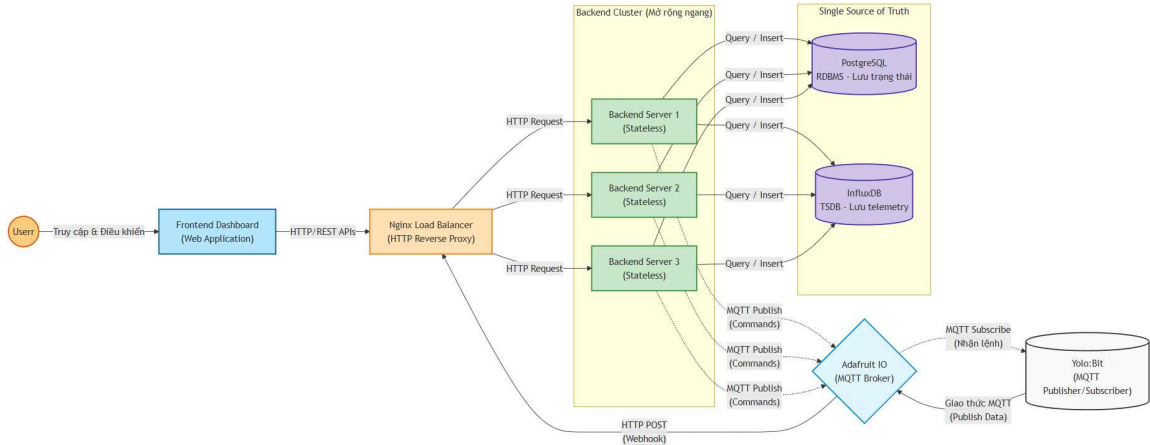
RESTful APIs and Webhooks (Client/Cloud to Backend)

- **Protocol:** HTTP/REST with strictly standardized JSON payloads.
- **Routing:** Prefixed with `/api/v1/` and routed through Nginx.

Core Endpoints:

- `POST /auth/login`: Authenticates user, returns JWT.
- `POST /adafruit-webhook`: Event-driven listener for real-time telemetry from Adafruit IO.
- `GET /telemetry`: Retrieves historical TSDB data.
- `POST /actuators/{id}/toggle`: Manually overrides an actuator state.
- `PUT /configurations`: Updates ideal and critical threshold ranges.
- `GET /alerts`: Fetches unresolved system and environmental alerts.

System Deployment Architecture



Deployment Architecture Overview

Distributed Deployment Strategy

- **Load Balancer (Nginx):** The single entry point receiving HTTP requests and Adafruit IO Webhooks. Uses Round Robin load balancing.
- **Stateless Backend Cluster:** Core application logic packaged into identical Docker containers. Allows easy horizontal scaling.
- **Single Source of Truth:** All instances connect to the centralized Hybrid Database Cluster to maintain data consistency.
- **Event-Driven IoT Integration:** Relies on Adafruit IO Webhooks instead of multiple servers polling data continuously.

Table of Contents

- 1 Introduction
- 2 System Architecture Overview
- 3 Requirements analysis
- 4 Software design
- 5 Implementation and Integration**
- 6 Conclusion

Edge Hardware Setup - Overview

Role of Edge Device in Smart Agriculture

- Collect real-time environmental metrics: temperature, humidity, and light intensity.
- Execute local feedback logic for fast responses without waiting for cloud processing.
- Display status and alerts directly on local indicators (LCD/OLED and RGB LED).
- Bridge sensor layer with cloud services through MQTT on Adafruit IO.

Practical impact

- Reduce latency for safety alerts.
- Maintain stable operation under intermittent connectivity.

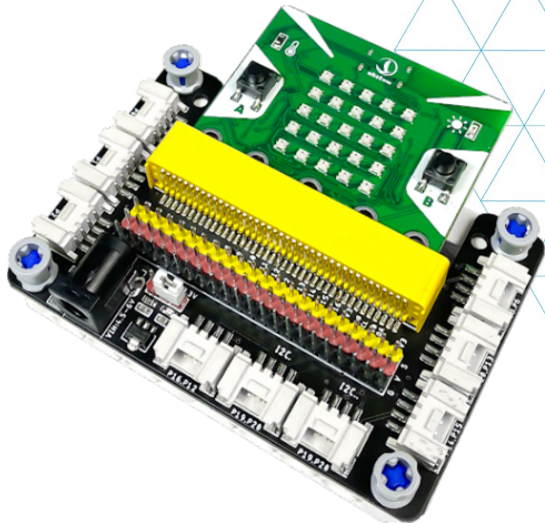
Edge Hardware Setup - Core Architecture

Core blocks

- Yolo:Bit (ESP32 dual-core) as edge controller.
- Extend Shield for I/O, I2C, and SPI expansion.
- Wi-Fi link for cloud communication via MQTT.
- Stable power by adapter/battery.

Why this setup

- Fast prototyping, easy wiring, simple maintenance.



Yolo:Bit + Extend Shield hardware block ▶

Edge Hardware Setup - Sensors & Actuators

Sensor Inputs

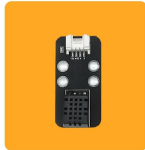
- **DHT20 (I2C)**: temperature and humidity sensing.
- **LDR (ADC)**: light intensity monitoring.
- **Infrared sensor**: intrusion/motion detection.

Design objective: close the loop between sensing, alerting, and actuation at edge level.

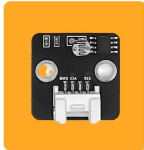
Indicators & Actuation

- **LCD/OLED**: local status and warning messages.
- **RGB LED**: green (stable), red (high temperature/alert).
- **Fan / Pump / Servo**: cloud-assisted environmental control.

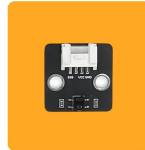
HW Sensors & Detail Images



Temp/Humidity



Light



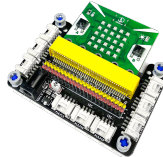
Infrared



LCD detail

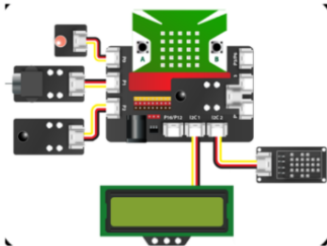


RGB detail

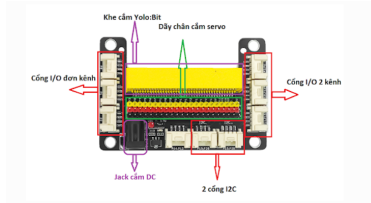


Yolo:Bit detail

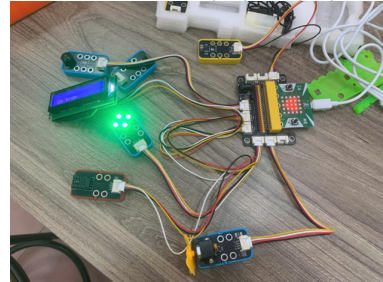
Edge Hardware Setup - HW Images



HW 1



HW 2



HW 3

Edge Hardware Setup - Data Flow (Edge to Cloud)

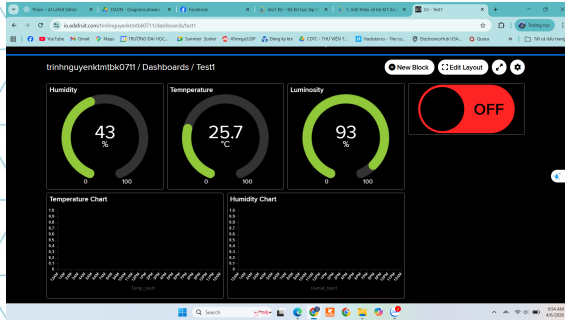
Operational pipeline

- ➊ **Setup phase:** initialize sensors, connect Wi-Fi, authenticate MQTT client.
- ➋ **Main loop (every 5s):** sample DHT20/LDR and apply local threshold logic.
- ➌ **Publishing:** push telemetry to Adafruit IO feeds.
- ➍ **Subscription:** receive actuator commands from backend services.

MQTT feeds (example)

- Telemetry: temp-test1, humid-test1, luminor-test1
- Control: actuator-fan, actuator-pump, actuator-led

Edge Hardware Setup - Dashboard & Feed



Dashboard overview

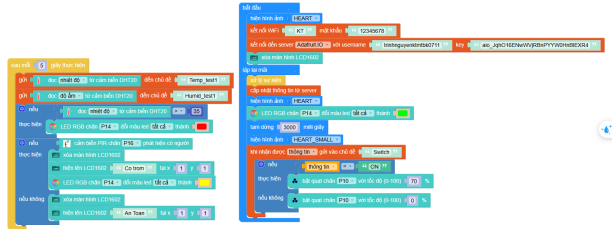
☐ Humid_test1	humid-test1	43.0	18 days ago	
☐ Luminor_test1	luminor-test1	93	18 days ago	
☐ Switch	switch	OFF	18 days ago	
☐ Temp_test1	temp-test1	25.7	18 days ago	

Feed monitoring interface

Edge Hardware Setup - Integration Highlights

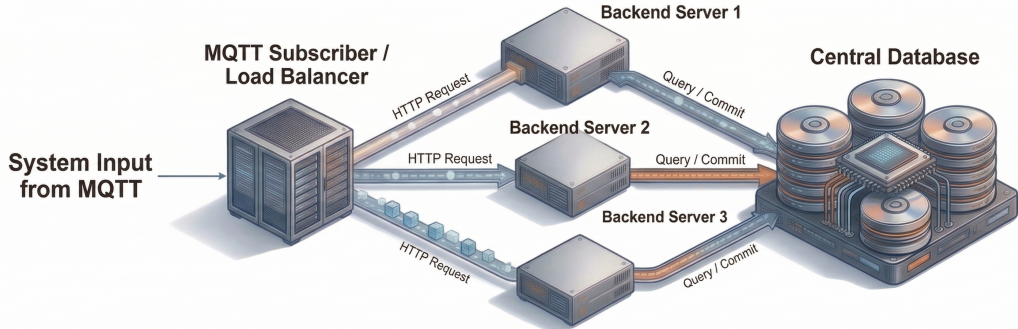
Key highlights

- MQTT + Adafruit IO for lightweight, stable telemetry.
- Local rule: high temperature triggers LCD + red LED alert.
- IR events are alerted locally before cloud confirmation.
- Reliable edge-to-cloud flow for dashboard automation.



Edge code stream (OhStem)

Load Balancer Architecture



MQTT Subscriber Overview

MQTT Basics

- **Protocol:** Lightweight messaging for IoT
- **Model:** Publish / Subscribe
- **Roles:**
 - **Publisher:** Sends data to a specific topic
 - **Subscriber:** Receives data from subscribed topics
 - **Broker:** Central server that routes messages
- **Features:**
 - QoS levels (0, 1, 2)
 - Persistent sessions
 - Retain messages

Example Subscriber Data

Subscriber receives a message from topic `sensor.temperature`:

```
{  
  "topic": "sensor.temperature",  
  "payload": {  
    "value": 25.5,  
    "unit": "C"  
  },  
  "qos": 1,  
  "retain": false,  
  "timestamp": "2026-04-07T07:00:00Z"  
}
```

Load Balancer Configuration

Configuration

- Implemented as **custom Node.js middleware**
- Acts as the **central traffic controller**
- Configuration controlled by:
upstream.conf
- Enables:
 - Flexible deployment
 - Easy scaling
 - Maintainable architecture

Subscriber Initialization

- Reads **adafruit_io** configuration
- Establishes:
 - Persistent MQTT connection
 - Authenticated session
- Subscribes to sensor topics:
 - Temperature
 - Humidity
- Converts received data into:
Standard JSON format

upstream.conf Structure

Configuration file divided into 3 main blocks:

- **http_port**
 - Defines listening port
 - Example: 8080
- **backends**
 - List of backend servers
 - Used for load distribution
- **adafruit_io**
 - MQTT connection settings
 - Topic subscriptions

```
{  
  "http_port": 8080,  
  "backends": [{  
    "host": "127.0.0.1",  
    "port": 3001  
  }, {  
    "host": "127.0.0.1",  
    "port": 3002  
  }],  
  "adafruit_io": {  
    "host": "mqtt://io.adafruit.com",  
    "username": "...",  
    "key": "...",  
    "topics": [  
      "sensor.temperature",  
      "sensor.humidity"  
    ]  
  }  
}
```

Round-Robin Load Balancing

Algorithm Overview

- Round-Robin: distribute requests sequentially among backends
- Simple, fair, ensures even load distribution
- Handles both:
 - External HTTP client requests
 - Internal MQTT messages from sensors
- Easy to implement in Node.js

Example Workflow

- 1 Backend list from `upstream.conf`:
 - Backend 1
 - Backend 2
 - Backend 3
- 2 First request → Backend 1
- 3 Second request → Backend 2
- 4 Third request → Backend 3
- 5 Repeat sequence for new requests

Benefits

- Prevents server overload, avoids single-point failure
- Scalable: easy to add new backends

Data Visualization

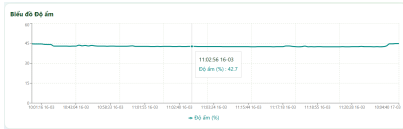


Figure 3: Humidity

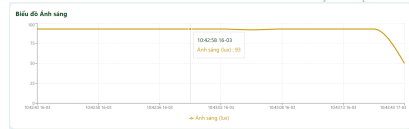


Figure 4: Light

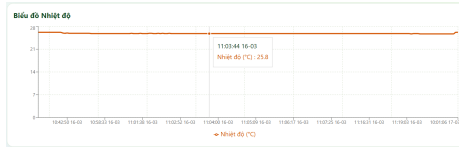


Figure 5: Temperature

- Interactive charts provide real-time and historical views of environmental metrics
- Each chart supports dynamic updates for better monitoring and analysis
- Help users quickly understand trends and current system conditions

Filtering & Aggregation

NÔNG NGHIỆP THÔNG MINH

Dashboard IoT Nông Trại

Theo dõi ánh sáng, nhiệt độ, độ ẩm theo thời gian và nhận cảnh báo sớm.

Làm mới ngay

Tự động cập nhật mỗi 5 phút
Lần gần nhất: 22:47:35 3/4/2026

Figure 6: Data refresh process

Figure 7 shows the filtering interface. It includes a dropdown menu for 'Khoảng lọc' (Filter range) with the selected option 'Mặc định - chi tiết' (Default - detail). There are two date selection fields: 'Từ ngày' (From date) and 'Đến ngày' (To date), both showing the format 'mm/dd/yyyy'. A 'Xuất CSV' (Export CSV) button is also present.

Figure 7: Filtering interface

- **Data Refresh**

- Manual refresh by user
- Auto update

- **Time Filtering**

- Day / Week / Month views
- Custom date range selection

- **Aggregation**

- Raw data for detail inspection
- Average values for trend analysis

- **Benefits**

- Reduce noise in sensor data
- Highlight long-term patterns

Status Evaluation & Alerts

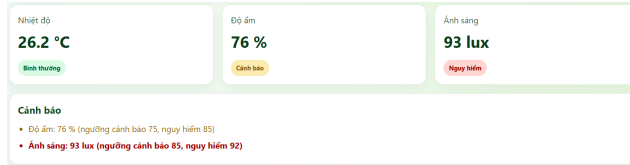


Figure 8: System alert visualization

- **System States:**

- Normal: within safe range
- Warning: approaching critical limits
- Critical: exceeds safe thresholds

- **Threshold Comparison:**

- Evaluate data in ideal and critical ranges

- **Alert Mechanism:**

- Dashboard notifications
- On-site RGB LED and LCD warnings

Table of Contents

- 1 Introduction
- 2 System Architecture Overview
- 3 Requirements analysis
- 4 Software design
- 5 Implementation and Integration
- 6 Conclusion**

Conclusion - Project Summary

Overview

- This project successfully implements a comprehensive smart agriculture system, integrating edge devices, cloud communication, backend services, and a web-based dashboard.

Key strengths

- **Real-time responsiveness:** Provides highly responsive real-time monitoring of environmental parameters.
- **Flexibility:** Supports both fully automated operation and manual intervention when needed.
- **Reliability:** Utilizes modern technologies such as Docker and Round-Robin Load Balancing, ensuring scalability and stable system performance.

Limitations and future improvements

- The integration between system modules is not yet fully consistent and requires further refinement.
- Additional features can be developed, such as weather-based recommendation systems and camera-based monitoring for detecting unusual objects in the garden.

- THE END -

Thank you for your attention

Acknowledgements:

I would like to express my sincere gratitude to my supervisor,
Mr. Diep Thanh Dang,
for his dedicated guidance and support throughout this project.

Contact: hoang.nguyen2311070@hcmut.edu.vn